NELC Compliance Features

Claude Code Implementation Plan

Local Development with Mock Services

This plan is designed for implementation WITHOUT paid third-party services. All Zoom features use mock services locally. When client provides Zoom credentials, swap mock for real with zero code changes to business logic.

Architecture: Mock Service Strategy

Every external service integration follows the same pattern: an interface/abstract class that both the real service and mock service implement. This allows local development without paid accounts, and a clean swap when real credentials arrive.

Service Interface Pattern

```
// zoom.interface.ts
export interface IZoomService {
  createMeeting(data: CreateMeetingDto): Promise<ZoomMeeting>;
  getParticipantReport(meetingId: string): Promise<Participant[]>;
}

// zoom.service.ts (REAL - uses Zoom API)
@Injectable()
export class ZoomService implements IZoomService { ... }

// zoom-mock.service.ts (MOCK - returns realistic fake data)
@Injectable()
export class ZoomMockService implements IZoomService { ... }

// zoom.module.ts - swap based on env
const provider = process.env.ZOOM_MOCK === 'true'
  ? ZoomMockService : ZoomService;
providers: [{ provide: 'IZoomService', useClass: provider }]
```

Rule: Business logic code NEVER imports ZoomService or ZoomMockService directly. It always injects IZoomService via the interface token. This makes the swap invisible.

Phase 1: NELC Audit Logs

100% local — No external dependencies. Start immediately.

Database Schema

```
// backends/src/database/schema/nelc-audit-logs.ts
export const nelcAuditLogs = pgTable('nelc_audit_logs', {
  id: uuid('id').primaryKey().defaultRandom(),
  eventType: varchar('event_type', { length: 60 }).notNull(),
  entityType: varchar('entity_type', { length: 40 }).notNull(),
  entityId: uuid('entity_id'),
  userId: uuid('user_id').references(() => users.id),
  metadata: jsonb('metadata'),
  ipAddress: varchar('ip_address', { length: 45 }),
  createdAt: timestamp('created_at').defaultNow(),
});

// Indexes:
// - eventType
// - (entityType, entityId) composite
// - userId
// - createdAt
```

Event Type Constants

```
export const NELC_EVENTS = {
  SESSION_STARTED: 'SESSION_STARTED',
  SESSION_ENDED: 'SESSION_ENDED',
  ATTENDANCE_MARKED_MANUAL: 'ATTENDANCE_MARKED_MANUAL',
  ATTENDANCE_AUTO_SYNCED: 'ATTENDANCE_AUTO_SYNCED',
  ATTENDANCE_ZOOM_JOIN: 'ATTENDANCE_ZOOM_JOIN',
  ATTENDANCE_ZOOM_LEAVE: 'ATTENDANCE_ZOOM_LEAVE',
  ENROLLMENT_CREATED: 'ENROLLMENT_CREATED',
  ENROLLMENT_LIMIT_ENFORCED: 'ENROLLMENT_LIMIT_ENFORCED',
  ASSISTANT_ALLOCATED: 'ASSISTANT_ALLOCATED',
  COMPLIANCE_CHECK_PASSED: 'COMPLIANCE_CHECK_PASSED',
  COMPLIANCE_CHECK_FAILED: 'COMPLIANCE_CHECK_FAILED',
  ZOOM_MEETING_CREATED: 'ZOOM_MEETING_CREATED',
  ZOOM_SYNC_COMPLETED: 'ZOOM_SYNC_COMPLETED',
} as const;
```

Service (Immutable — NO update/delete)

```
// backends/src/modules/nelc-audit/nelc-audit.service.ts
@Injectable()
export class NelcAuditService {
  async log(event: {
    eventType: string; entityType: string;
    entityId?: string; userId?: string;
```

```
    metadata?: Record<string, any>; ipAddress?: string;
  }): Promise<void> { /* INSERT only */ }

  async query(filters: {
    eventType?: string; entityType?: string;
    entityId?: string; userId?: string;
    startDate?: Date; endDate?: Date;
    page?: number; limit?: number;
  }): Promise<PaginatedResult> { /* SELECT only */ }

  async getSummary(startDate: Date, endDate: Date):
    Promise<{ eventType: string; count: number }[]> { }
}
```

API Endpoints (Admin-only)

Method	Route	Description	Role
GET	/nelc-audit	Paginated, filterable audit logs	ADMIN
GET	/nelc-audit/summary	Aggregated counts by event type for date range	ADMIN

Files to Modify

- **backends/src/database/schema/index.ts —** Add export * from './nelc-audit-logs'
- **backends/src/app.module.ts —** Add NelcAuditModule (global: true)
- **backends/src/modules/sessions/sessions.service.ts —** Inject NelcAuditService, log in start(), end(), cancel()
- **backends/src/modules/enrollments/enrollments.service.ts —** Log in markAttendance(), bulkMarkAttendance()
- **frontend/src/lib/api.ts —** Add nelcAuditApi.getLogs() and .getSummary()

Verification: Run npx drizzle-kit generate && npx drizzle-kit push. Test GET /nelc-audit with curl. Confirm immutability (no PATCH/DELETE endpoints exist).

Phase 2: Learner Limits Enforcement

100% local — Pure business logic + database. No external dependencies.

NELC Rules (Configurable)

```
// backends/src/modules/courses/nelc-config.ts
// IMPORTANT: These should be in a settings table,
// not hardcoded. Admin can update via dashboard.
export const NELC_LIMITS = {
  LONG_COURSE_DAYS_THRESHOLD: 5,
  LONG_COURSE_MAX_LEARNERS: 40,
  SHORT_COURSE_MAX_LEARNERS: 50,
};

export function classifyCourse(startDate: Date, endDate: Date) {
  const days = differenceInCalendarDays(endDate, startDate);
  return days > NELC_LIMITS.LONG_COURSE_DAYS_THRESHOLD
    ? 'long' : 'short';
}

export function getNelcMaxEnrollment(classification: string) {
  return classification === 'long'
    ? NELC_LIMITS.LONG_COURSE_MAX_LEARNERS
    : NELC_LIMITS.SHORT_COURSE_MAX_LEARNERS;
}
```

Enrollment with Limit Check

```
// In enrollments.service.ts
async createEnrollment(courseId, studentId, user) {
  const course = await this.getCourse(courseId);
  const count = await this.getActiveEnrollmentCount(courseId);
  const classification = classifyCourse(course.startDate, course.endDate);
  const maxAllowed = course.maxEnrollment
    ?? getNelcMaxEnrollment(classification);

  if (count >= maxAllowed) {
    await this.auditService.log({
      eventType: 'ENROLLMENT_LIMIT_ENFORCED',
      entityType: 'course', entityId: courseId,
      metadata: { count, maxAllowed, classification },
    });
    throw new BadRequestException(
      `NELC limit reached: ${count}/${maxAllowed} for ${classification} course`
    );
  }

  // Use DB transaction with row-level lock
  return await this.db.transaction(async (tx) => {
    // SELECT ... FOR UPDATE on course row
```

```
    // Re-check count inside transaction
    // INSERT enrollment
    // Audit log: ENROLLMENT_CREATED
  });
}
```

Frontend Changes

- **api.ts —** Add enrollStudent(courseId, { studentId }) method
- **Course cards —** Show enrollment capacity bar (enrolled/max) with color coding
- **Session detail —** Display NELC limit info callout when near capacity
- **Create course form —** Auto-calculate max enrollment based on dates; allow manual override

Verification: Create a course with dates. Enroll students up to the limit. Verify next enrollment is rejected with proper NELC error message. Check audit log has ENROLLMENT_LIMIT_ENFORCED entry.

Phase 3: Auto Assistant Allocation

100% local — Pure business logic + database.

Database Schema

```
// backends/src/database/schema/session-assistants.ts
export const sessionAssistants = pgTable('session_assistants', {
  id: uuid('id').primaryKey().defaultRandom(),
  sessionId: uuid('session_id').references(() => sessions.id, { onDelete:
'cascade' }),
  userId: uuid('user_id').references(() => users.id),
  role: varchar('role', { length: 20 }), // 'co_host' | 'assistant'
  assignedAt: timestamp('assigned_at').defaultNow(),
  assignedBy: uuid('assigned_by').references(() => users.id),
  isAutoAssigned: boolean('is_auto_assigned').default(false),
}, (table) => ({
  uniqueSessionUser: unique().on(table.sessionId, table.userId),
}));
```

NELC Allocation Rules

Enrolled Learners	Required Assistants	Action
≤25	0	No assistant needed
26–35	1	Auto-assign 1 available instructor
>35	2	Auto-assign 2 available instructors

These thresholds need NELC confirmation from client. Currently configurable in code.

API Endpoints

Method	Route	Description	Role
GET	/sessions/:id/assistants	List assigned assistants	INSTRUCTOR, ADMIN, MANAGER
POST	/sessions/:id/assistants	Manual assign	ADMIN, MANAGER
DELETE	/sessions/:id/assistants/:userId	Remove assistant	ADMIN, MANAGER

Phase 4: Real-time Zoom Attendance (with Mock)

LOCAL DEV: Uses ZoomMockService that simulates webhook events. PRODUCTION: Swap to real ZoomService when client provides credentials.

Database Schema

```
// backends/src/database/schema/attendance-logs.ts
export const attendanceLogs = pgTable('attendance_logs', {
  id: uuid('id').primaryKey().defaultRandom(),
  sessionId: uuid('session_id').references(() => sessions.id, { onDelete:
'cascade' }),
  studentId: uuid('student_id').references(() => users.id, { onDelete:
'cascade' }),
  eventType: varchar('event_type', { length: 20 }), // 'join' | 'leave'
  timestamp: timestamp('timestamp').notNull(),
  zoomParticipantId: varchar('zoom_participant_id', { length: 100 }),
  zoomUserEmail: varchar('zoom_user_email', { length: 255 }),
  rawPayload: jsonb('raw_payload'),
});
```

Mock Webhook Service

```
// backends/src/modules/zoom/zoom-mock-webhook.service.ts
@Injectable()
export class ZoomMockWebhookService {
  /**
   * Simulates Zoom webhook events for local testing.
   * Call via: POST /zoom/mock/simulate
   * Body: { sessionId, studentEmail, event: 'join'|'leave' }
   */
  async simulateParticipantEvent(data) {
    const payload = this.buildMockPayload(data);
    return this.webhookService.handleEvent(payload);
  }

  /**
   * Simulates a full session: join all enrolled students,
   * wait N minutes, leave some early, end session.
   */
  async simulateFullSession(sessionId) {
    // 1. Get enrolled students
    // 2. Simulate joins with 0-5 min random delay
    // 3. Simulate 2-3 early leaves
    // 4. Simulate session end (all remaining leave)
    // Returns: { joined: N, earlyLeaves: N, duration: Nmin }
  }
}
```

Mock-Only Endpoints (dev environment)

Method	Route	Description
POST	/zoom/mock/simulate	Simulate a single join/leave event
POST	/zoom/mock/full-session/:sessionId	Simulate an entire session with realistic attendance
GET	/zoom/mock/status	Check if mock mode is active

CRITICAL: Mock endpoints are guarded by NODE_ENV check. They do NOT exist in production builds. Add guard: if (process.env.NODE_ENV === 'production') throw new ForbiddenException();

Real Webhook Endpoints (production)

Method	Route	Description
POST	/zoom/webhook	Receives all Zoom webhook events (participant_joined, participant_left, meeting.ended)

The webhook controller verifies the x-zm-signature header before processing. In mock mode, this verification is skipped.

Add zoomEmail to users table NOW

IMPORTANT: Add a zoomEmail field to the users table in this phase. Students may use different emails in Zoom vs the platform. Without this, attendance matching will fail.

```
// Modify: backends/src/database/schema/users.ts
zoomEmail: varchar('zoom_email', { length: 255 }),

// Matching logic in webhook handler:
// 1. Try exact email match with user.email
// 2. Try exact match with user.zoomEmail
// 3. Log unmatched participant for manual resolution
```

Phase 5: Post-Meeting Attendance Sync

LOCAL DEV: Mock service returns realistic participant report data. PRODUCTION: Uses Zoom Reports API (requires Business plan).

Mock Report Service

```
// backends/src/modules/zoom/zoom-mock-report.service.ts
@Injectable()
export class ZoomMockReportService implements IZoomReportService {
  async getParticipantReport(meetingId: string) {
    // Returns realistic mock data:
    return {
      participants: [
        {
          id: 'mock-p1',
          name: 'Ahmed Al-Farsi',
          user_email: 'ahmed@example.com',
          join_time: '2025-02-19T09:02:00Z',
          leave_time: '2025-02-19T10:58:00Z',
          duration: 116, // minutes
          attentiveness_score: 'Good',
        },
        // ... generates for all enrolled students
      ],
    };
  }
}
```

Sync Logic

The sync service reconciles Zoom data with existing attendance records:

- **Rule 1:** Manual marks always win over Zoom data (instructor override).
- **Rule 2:** Zoom data supplements missing records (students who forgot to check in).
- **Rule 3:** Duration is recalculated from all join/leave pairs.
- **Rule 4:** Unmatched Zoom participants are logged for manual resolution.

Cron Job (Production)

```
@Cron('*/10 * * * *') // Every 10 minutes
async syncRecentlyEndedSessions() {
  // 1. Find sessions: endedAt < now - 10min AND zoomSyncedAt IS NULL
  // 2. For each: fetch Zoom report, reconcile, update
  // 3. Set zoomSyncedAt = now()
  // 4. Audit log: ZOOM_SYNC_COMPLETED
}
```

Schema Change

```
// Modify: backends/src/database/schema/sessions.ts
zoomSyncedAt: timestamp('zoom_synced_at'),
```

API Endpoints

Method	Route	Description	Role
POST	/sessions/:id/sync-zoom	Manual sync trigger	INSTRUCTOR, ADMIN
GET	/sessions/:id/sync-status	Check sync status	INSTRUCTOR, ADMIN

Phase 6: NELC Compliance Dashboard

100% local — Reads from data created in Phases 1-5. Pure aggregation layer.

Compliance Checks

Check	Logic	Status
Enrollment Limits	enrolledCount <= maxAllowed	GREEN if under, RED if over
Assistant Allocation	assignedAssistants >= requiredAssistants	GREEN if met, AMBER if 1 short, RED if 2+ short
Zoom Settings	waiting_room: true, auth required	GREEN if correct settings, RED if missing
Attendance Compliance	% of students meeting 70% threshold	GREEN >90%, AMBER 70-90%, RED <70%
Audit Trail	All critical events logged in nelc_audit_logs	GREEN if complete, RED if gaps detected

API Endpoints

Method	Route	Description	Role
GET	/nelc-compliance/session/:id	Single session compliance	INSTRUCTOR, ADMIN, MANAGER
GET	/nelc-compliance/course/:id	Course aggregate compliance	ADMIN, MANAGER
GET	/nelc-compliance/summary	Dashboard-level summary	ADMIN, MANAGER

Frontend Integration

- **NELCComplianceWidget —** Wire existing widget in Course Manager dashboard to real GET /nelc-compliance/summary
- **Session cards —** Add green/amber/red compliance indicator dot
- **Pre-start check —** Modal when instructor clicks 'Start Session' showing compliance status
- **Admin detail page —** New /admin/nelc-compliance route with full course drill-down table

Environment Configuration

```
# .env.development (LOCAL - mock everything)
ZOOM_MOCK=true
ZOOM_CLIENT_ID=not-needed-in-mock
ZOOM_CLIENT_SECRET=not-needed-in-mock
ZOOM_WEBHOOK_SECRET=local-dev-secret
COMMS_PROVIDER=mock  # email-only in dev

# .env.staging (REAL Zoom, test data)
ZOOM_MOCK=false
ZOOM_CLIENT_ID=<from-client>
ZOOM_CLIENT_SECRET=<from-client>
ZOOM_WEBHOOK_SECRET=<from-zoom-marketplace>
ZOOM_ACCOUNT_ID=<from-client>
COMMS_PROVIDER=mock

# .env.production
ZOOM_MOCK=false
ZOOM_CLIENT_ID=<real>
ZOOM_CLIENT_SECRET=<real>
ZOOM_WEBHOOK_SECRET=<real>
ZOOM_ACCOUNT_ID=<real>
COMMS_PROVIDER=unifonic  # or infobip
```

Build Verification Checklist

After EACH phase, run:

```
# 1. Apply schema changes
npx drizzle-kit generate && npx drizzle-kit push

# 2. Verify zero TypeScript errors
cd backends && npm run build
cd frontend && npm run build

# 3. Test endpoints
# Phase 1: curl GET /nelc-audit
# Phase 2: curl POST /courses/:id/enroll (test limit)
# Phase 3: curl GET /sessions/:id/assistants
# Phase 4: curl POST /zoom/mock/simulate
# Phase 5: curl POST /sessions/:id/sync-zoom
# Phase 6: curl GET /nelc-compliance/summary
```

Key Risks & Mitigations

Risk	Impact	Mitigation
Zoom email mismatch	Students use different email in Zoom vs platform	Add zoomEmail field in Phase 4. Implement fuzzy matching as fallback.

Risk	Impact	Mitigation
Race condition on enrollment	Concurrent requests bypass limit	Use DB transaction with SELECT FOR UPDATE row-level lock.
Zoom webhook delivery failure	Missed join/leave events	Phase 5 sync is the safety net. Add health check alert if no webhooks for 2+ hours.
Zoom Reports API delay	Reports available 5-10 min after meeting	Cron runs every 10 min. Manual sync button for impatient instructors.
NELC rule changes	Hardcoded limits become wrong	Store limits in settings table, editable via Admin dashboard.